1. Project Overview

This project analyzes a real-world water pump sensor dataset containing multiple sensor measurements and Remaining Useful Life (RUL) values. The objective is to apply algorithmic techniques to detect patterns in machine behavior and examine how these patterns relate to machine health.

Instead of using machine learning models, this project focuses on algorithm design and implementation. Three algorithms were implemented from scratch:

- Divide-and-Conquer Segmentation
- Divide-and-Conquer Clustering
- Maximum Subarray (Kadane's Algorithm)

The results of these algorithms are compared with machine-health categories derived from RUL values, allowing us to investigate how sensor behavior changes across different stages of machine degradation.

2. Installation and Usage Instructions

The project is implemented using Python 3.

Required libraries:

- pandas
- numpy
- matplotlib

1. Install dependencies:
    a. pip install pandas numpy matplotlib

2. Download the dataset:

   https://www.kaggle.com/datasets/anseldsouza/water-pump-rul-predictive-maintenance

3. Place the dataset file in the same directory as the program.
4. Run the program

Program workflow:

1. Load and preprocess dataset
2. Convert RUL values into machine-condition categories
3. Execute Task 1, Task 2, and Task 3
4. Output summary tables and segmentation plots

3. Code Structure Explanation

The code is organized into several sections that correspond to the major steps of the project.

- **Data Loading**
  - Dataset loaded using pandas
  - Only the first 10,000 rows are used
  - Sensor columns are detected automatically using the prefix “sensor_”
- **Data Cleaning**
  - Convert values to numeric
  - Handle missing values using forward fill and backward fill
- **RUL Category Transformation**

  Continuous RUL values are converted into four machine-health categories using quantiles.

- **Algorithm Tasks**

  The program then executes three algorithmic tasks:

  - Task 1 – Segmentation
  - Task 2 – Clustering
  - Task 3 – Maximum deviation detection

  Each task generates summary outputs used for analysis.

4. Algorithm Descriptions

- **Divide-and-Conquer Segmentation**

  The segmentation algorithm recursively divides a time-series signal into smaller segments based on signal variance.

  Algorithm steps:

  1. Compute variance of the current segment.
  2. If variance exceeds a threshold, split the segment into two halves.
  3. Recursively repeat the process for each half.
  4. Stop splitting when:
     a. segment size becomes small, or
     b. variance falls below the threshold.

  The threshold used in this project is half of the variance of the full signal.

  The algorithm produces a set of segments representing stable regions of the signal.

  The Segmentation Complexity Score is defined as the number of resulting segments.

- **Divide-and-Conquer Clustering**

  A custom top-down clustering algorithm was implemented to group similar machine states.

  Algorithm steps:

  1. Start with all points in one cluster.
  2. Compute Sum of Squared Errors (SSE) for each cluster.
  3. Select the cluster with the largest SSE.
  4. Identify the sensor feature with the largest variance.
  5. Split the cluster using the median of that feature.
  6. Continue splitting until four clusters are created.

  Each cluster represents a group of machine states with similar sensor measurements.

- **Maximum Subarray (Kadane)**

  Kadane's algorithm is used to detect time intervals with the strongest sustained deviation in sensor signals.

  Signal preprocessing:

$$d[i] = | sensor[i] - sensor[i-1] |$$

$$x[i] = d[i] - mean(d)$$

  This transformation highlights abnormal deviations in the signal.

  Kadane's algorithm scans the array and identifies the maximum sum contiguous interval, representing the strongest sustained deviation in the sensor signal.

5. Toy Example Verification

- **Segmentation Toy Example**

A simple signal was used to test the recursive segmentation algorithm.

Input signal: [1, 1, 1, 10, 11, 12, 1, 1]

This signal contains a stable region, followed by a region with larger values, and then returns to a stable region. The segmentation algorithm should detect the change in variance and split the signal into multiple segments.

Algorithm output:

```
Segmentation validation:
Toy Segmentation Output:
[(0, 2), (2, 3), (3, 4), (4, 6), (6, 8)]
```

Figure a - Segmentation Toy Example Validation Output

The output represents the start and end indices of the segments detected by the algorithm. The signal is divided into smaller regions where the variance condition changes.

- **Clustering Toy Example**

A small set of points was used to test the top-down clustering algorithm.

Input points: (1,1), (2,1), (8,8), (9,8)

These points clearly form two groups: one near (1,1) and another near (8,8).

Algorithm output:

```
Clustering validation:
Toy clustering output:
Cluster 0 [[1 1]
 [2 1]]
Cluster 1 [[8 8]
 [9 8]]
```

Figure b - Clustering Toy Example Validation Output

The clustering algorithm successfully separates the points into two clusters based on their distance and variance.

- **Kadane Toy Example**

Kadane's algorithm was tested using a classic example array.

Input array: [-2, 1, -3, 4, -1, 2, 1, -5, 4]

Algorithm output:

```
Kadane validation:
Toy Kadane Output: (np.int64(6), 3, 6)
```

Figure c - Kadane Toy Example Validation Output

This output represents:

- Maximum sum = 6
- Start index = 3
- End index = 6

The corresponding maximum subarray is:

[4, -1, 2, 1]

This confirms that Kadane's algorithm correctly identifies the interval with the largest sum.

6. Dataset Description

The dataset used in this project is Water Pump RUL – Predictive Maintenance, which contains time-series sensor measurements collected from an industrial water pump machine. The dataset includes multiple sensor signals that capture machine behavior over time, along with the Remaining Useful Life (RUL) of the machine.

Each row in the dataset represents one time instance of machine operation. For this project, only the first 10,000 rows of the dataset were used.

The dataset contains 52 sensor channels, a timestamp column, and the RUL value measured in hours.

- **Variables used**

  The analysis focuses on the sensor signals and the RUL variable.

Variable	**Description**
timestamp	Time index of the sensor measurement
sensor_00 – sensor_51	Sensor measurements describing machine behavior
rul	Remaining Useful Life of the machine (in hours)

In the program, the sensor variables were automatically detected by selecting columns whose names start with “sensor_”. These sensor values were used as the input features for the segmentation, clustering, and deviation detection algorithms.

The variable rul was used to derive machine condition categories that allow comparison between algorithm results and machine health states.

- **RUL → 4-category transformation**

The continuous RUL values were converted into four machine-health categories using quantile thresholds. This transformation allows the algorithms to evaluate machine condition using discrete states.

The following quantiles were computed from the dataset:

Q10 – 10% quantile of RUL

Q40 – 40% quantile of RUL

Q90 – 90% quantile of RUL

Based on these thresholds, the RUL values were divided into four categories:

Condition	**Definition**
Extremely Low RUL	rul < Q10
Moderately Low RUL	Q10 ≤ rul < Q40
Moderately High RUL	Q40 ≤ rul < Q90
Extremely High RUL	rul ≥ Q90

These four categories represent different stages of machine health. Machines with extremely low RUL are close to failure, while machines with extremely high RUL are in earlier stages of their lifecycle.

This categorical representation allows the segmentation, clustering, and maximum deviation algorithms to be compared with machine degradation stages.

7. Execution Results

- **Task1**

Ten sensors were randomly selected from the available 52 sensors and analyzed using the recursive segmentation algorithm. For each selected sensor, the algorithm divided the time series into segments based on variance.

The algorithm produced a Segmentation Complexity Score, which represents the number of segments generated for the signal.

A higher complexity score indicates that the sensor signal fluctuates more frequently, while a lower score suggests that the signal is more stable.

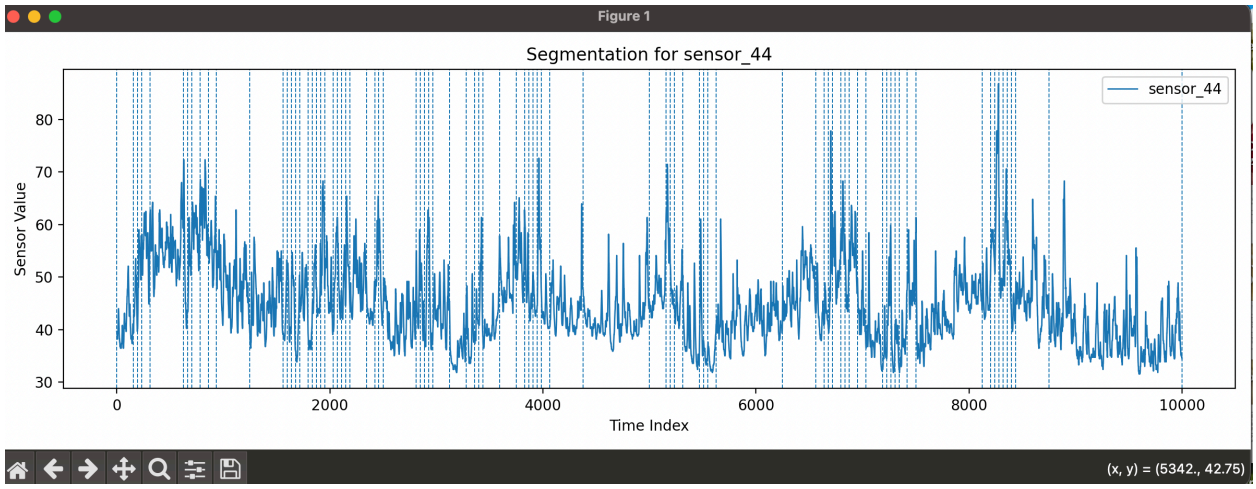

Figure d - Segmentation Plot (sensor_44)

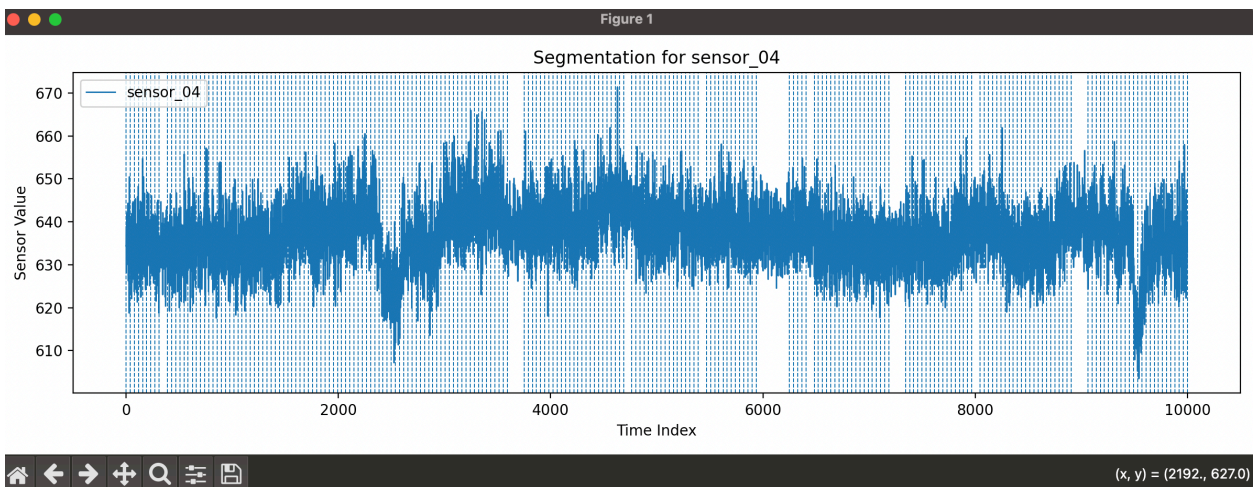

Figure e - Segmentation Plot (sensor_04)

```
Task 1 Summary (Segmentation Complexity Score):
----------------------------------------
Sensor       Complexity Score
----------------------------------------
sensor_44    85
sensor_04    235
sensor_21    13
sensor_47    61
sensor_00    14
sensor_14    48
sensor_37    48
sensor_29    2
sensor_25    10
sensor_06    31
```

Figure f - Task 1 Segmentation Complexity Summary

Step 5 Discussion – Relation to RUL Categories

For each segment produced by the segmentation algorithm, the dominant RUL category was identified. This allowed us to analyze whether unstable signal segments correspond to particular machine health states.

In several sensors, segments with larger signal fluctuations were associated with lower RUL categories, particularly Moderately Low RUL and Extremely Low RUL. This suggests that machine behavior becomes more unstable as the system approaches failure.

Segments associated with Extremely High RUL typically showed more stable signal behavior, indicating normal machine operation.

Step 6 Discussion – Temporal Dynamics of Time Series

The segmentation results provide insight into the temporal dynamics of each sensor signal.

Sensors with higher segmentation complexity scores show frequent changes in variance over time. These sensors likely capture dynamic machine behaviors such as pressure changes or vibration fluctuations.

Sensors with lower complexity scores appear more stable and may represent background operating conditions. Comparing complexity scores across sensors helps identify which sensors are most sensitive to changes in machine behavior.

- **Task2**

The top-down clustering algorithm was applied to the full dataset consisting of 10,000 rows and 52 sensor measurements. The algorithm recursively split the dataset until four clusters were produced.

Each cluster groups time instances with similar sensor measurements.

For each cluster the following statistics were computed:

- cluster size
- dominant RUL category
- majority class count

```
================ TASK 2 ================

Task 2 Summary (Majority Class Count per Cluster):
--------------------------------------------------------------------------------
Cluster    Size       Majority Class            Majority Count
--------------------------------------------------------------------------------
0          3019       Moderately High RUL       1448
1          1989       Moderately High RUL       947
2          2497       Moderately High RUL       1578
3          2495       Moderately Low RUL        1347
```

Figure g - Task 2 Clustering Summary

Step 3 Discussion – Mapping Between Clusters and RUL Classes

To evaluate the clustering results, each cluster was compared with the four RUL categories.

In several clusters, a particular RUL category appeared as the dominant class. This indicates that certain combinations of sensor values correspond to specific machine health states.

Clusters dominated by Extremely Low RUL or Moderately Low RUL likely represent machine states closer to failure. Clusters dominated by Moderately High RUL or Extremely High RUL likely represent normal or early lifecycle machine states.

Although clustering does not directly use RUL values, the partial alignment between clusters and RUL categories suggests that sensor measurements capture meaningful information about machine condition.

- **Task3**

Kadane's algorithm was applied to each of the 52 sensors to identify time intervals with the strongest sustained deviations in sensor behavior.

For each sensor:

1. The absolute first difference of the signal was computed.
2. The mean of the differences was subtracted to highlight deviations.
3. Kadane's algorithm was used to find the interval with the maximum deviation sum.

The following values were recorded for each sensor:

- maximum deviation sum
- start index
- end index
- dominant RUL category during the interval

```
================ TASK 3 ================

Task 3 Summary (Kadane Maximum Deviation Intervals):
--------------------------------------------------------------------------------
Sensor       Max Deviation      Start      End        Dominant RUL Class
--------------------------------------------------------------------------------
sensor_31    16526.5259         830        4076       Moderately High RUL
sensor_27    10840.4786         334        623        Extremely High RUL
sensor_30    10405.2304         882        2443       Moderately High RUL
sensor_32    5165.2726          4197       8180       Moderately Low RUL
sensor_29    4935.5236          5061       9999       Moderately Low RUL
sensor_17    4372.1262          7630       9999       Moderately Low RUL
sensor_37    4352.5241          8808       9999       Extremely Low RUL
sensor_25    4152.6891          5332       9998       Moderately Low RUL
sensor_36    4095.0803          3545       9999       Moderately Low RUL
sensor_24    3682.6298          8600       9999       Extremely Low RUL
sensor_33    3681.7997          241        5543       Moderately High RUL
sensor_28    3619.9786          242        994        Extremely High RUL
sensor_35    2681.1193          3450       9998       Moderately Low RUL
sensor_21    1938.0375          934        6526       Moderately High RUL
sensor_40    1776.4385          7332       9113       Moderately Low RUL
sensor_34    1761.8507          2026       6660       Moderately High RUL
sensor_48    1321.9175          5586       7603       Moderately Low RUL
sensor_49    1271.9618          19         6923       Moderately High RUL
sensor_22    1209.9936          8599       9998       Extremely Low RUL
sensor_45    1190.3668          2170       3141       Moderately High RUL
```

Figure h - Task 3 Kadane Deviation Summary

sensor_23	1067.3180	267	3557	Moderately High RUL
sensor_46	1056.7493	7268	9783	Moderately Low RUL
sensor_04	1042.7335	736	5915	Moderately High RUL
sensor_13	1024.1241	5857	9975	Moderately Low RUL
sensor_16	927.4997	2018	4015	Moderately High RUL
sensor_43	880.2767	7337	9300	Moderately Low RUL
sensor_26	878.9493	273	2640	Moderately High RUL
sensor_05	859.8427	102	4790	Moderately High RUL
sensor_19	687.9851	8717	9999	Extremely Low RUL
sensor_51	630.0249	4577	7869	Moderately Low RUL
sensor_20	419.3834	1797	5399	Moderately High RUL
sensor_44	411.0240	172	3130	Moderately High RUL
sensor_14	354.0737	8622	9995	Extremely Low RUL
sensor_47	325.6725	4852	9897	Moderately Low RUL
sensor_10	313.1901	2786	9995	Moderately High RUL
sensor_38	276.1320	3110	9843	Moderately Low RUL
sensor_01	184.0050	1	2039	Moderately High RUL
sensor_39	154.8466	8222	9562	Moderately Low RUL
sensor_41	139.9432	2868	9938	Moderately High RUL
sensor_12	108.9272	1870	2960	Moderately High RUL
sensor_42	100.3629	7326	9894	Moderately Low RUL
sensor_11	97.0692	1835	7863	Moderately High RUL
sensor_09	61.4696	5832	5920	Moderately High RUL
sensor_18	46.3140	7904	9995	Moderately Low RUL
sensor_03	32.3386	5217	9489	Moderately Low RUL
sensor_08	24.5149	5767	8750	Moderately Low RUL
sensor_02	19.5738	8990	9826	Extremely Low RUL
sensor_07	11.3652	3884	9997	Moderately Low RUL
sensor_00	9.4939	62	4831	Moderately High RUL
sensor_06	7.6562	2689	4489	Moderately High RUL

Figure i - Task 3 Kadane Deviation Summary

Step 5 Discussion – Sensors as Early Indicators of Low RUL

The deviation intervals identified by Kadane's algorithm represent periods where sensor signals change significantly. Such changes may indicate abnormal machine behavior.

Sensors whose maximum deviation intervals occur primarily during low RUL categories may act as early indicators of machine degradation. These sensors may detect physical changes in the system before failure occurs.

By comparing deviation intervals with RUL categories, it becomes possible to identify sensors that are particularly sensitive to changes in machine condition. These sensors could be useful for predictive maintenance monitoring systems.

8. Discussion and Conclusions

- **Findings interpretation**

The results of the three algorithmic tasks show that classical algorithms can reveal meaningful patterns in time-series sensor data. The Divide-and-Conquer segmentation algorithm successfully identified stable and unstable regions in sensor signals. Sensors with higher segmentation complexity scores showed more frequent signal fluctuations, suggesting that they capture more dynamic machine behavior.

The clustering results indicate that groups of sensor measurements can represent different machine states. Even though the clustering algorithm did not directly use RUL values, several clusters were dominated by specific RUL categories. This suggests that sensor patterns contain information related to machine health and degradation stages.

The maximum subarray analysis using Kadane's algorithm identified intervals where sensor signals experienced strong sustained deviations. In several cases, these deviation intervals corresponded to lower RUL categories. This indicates that large signal deviations may occur when the machine approaches failure, suggesting that certain sensors could serve as early indicators of machine degradation.

Overall, the results demonstrate that algorithmic techniques can provide useful insights into machine behavior and help identify patterns related to machine health.

- **Challenge in this project**

  One challenge in this project was handling the large time-series dataset while implementing the algorithms from scratch. Processing 10,000 rows and 52 sensor signals required careful data preparation and efficient algorithm design.

  Another challenge was selecting appropriate parameters for the segmentation algorithm, such as the variance threshold and minimum segment size. If the threshold was too small, the algorithm produced too many segments, while a larger threshold produced fewer segments.

  Interpreting the sensor behavior was also challenging because the dataset does not provide detailed descriptions of what each sensor measures. As a result, understanding the physical meaning behind sensor fluctuations required relying primarily on statistical patterns rather than domain knowledge.

- **Limitations and possible improvements**

  Although the algorithms provided useful insights, the project has several limitations. The segmentation algorithm analyzes each sensor signal independently and does not consider relationships between different sensors. Similarly, the clustering algorithm groups data points based only on sensor measurements without incorporating temporal information. Another limitation is that the segmentation threshold was fixed for all sensors, which may not be optimal for signals with different variance levels.

  Possible improvements include implementing adaptive thresholds for segmentation, incorporating correlations between multiple sensors, and using time-series anomaly detection techniques. Future work could also compare these algorithmic approaches with

machine learning models to evaluate whether more advanced methods improve predictive maintenance performance.